Software Cost Estimation

Software cost estimation is the ***process of predicting the effort*** required to develop a software system. Software measurement, just like any other measurement in the physical world, can be categorized into ***Direct measures and Indirect measures***.

Direct measures of the software process include measurement of cost and effort applied. Direct measures of a product also include Line Of Code (LOC) produced, execution speed, memory size, and defects reported over some of times.

Indirect measures of the software process include measurement of resulting features of the software. Indirect measures of the product also include its functionality, quality, complexity, usability etc.

On the other word, estimation is ***not an exact science***. Too many variables – human, technical, and environment – can affect the ultimate cost of software and effort applied to develop it.

In order to make a reliable cost and effort estimation, there are a lot of techniques that provide estimates with acceptable amount of risk.

These methods can be categorized into-

a) Decomposition techniques
    I) **L**ines **O**f **C**ode (LOC)
    II) **F**unctional **P**oint (FP) Estimation
b) Empirical Estimation models
    I) **CO**nstructive **CO**st **MO**del (COCOMO)
        1. Basic COCOMO
        2. Intermediate COCOMO

3. Advanced COCOMO

Line Of Code (LOC)

LOC is a direct approach method and requires a higher level of detail by means of decomposition and partitioning. From statement of software scope, software is decomposed into problem functions that can each be estimated individually. LOC is then calculated for each function.

An expected value is then computed using the following formula-

$$EV = (S_{opt} + 4S_m + S_{pess}) / 6$$

where,

EV stands for the Estimation Variable

S_{opt} stands for Optimistic Estimation case when everything goes right

S_m stands for the most likely Estimation case given normal problems and opportunities

S_{pess} stands for the pessimistic Estimation case where everything goes wrong

Once the expected value for estimation variable has been determined, historical LOC or FP data are applied and person months, costs etc. are calculated using the following formula

Productivity = KLOC / Person-month
Quality = Defects / KLOC

Cost = $/LOC
Documentation = pages of documentation / KLOC

Where,
KLOC stand for number of code (in thousands)
Person-month stand for is the time (in months) taken by developers to finish the product
Defects stand for Total Number of errors discovered.

Example:

Problem Statement: Take our Library management system case. Software developed for library will accept data from operator for issuing and returning books. Issuing and returning will require some validity checks. For issue it is required to check if the member has already issued the maximum books allowed. In case for return, if the member is returning the book after the due date then fine has to be calculated. All the interactions will be through user interface. Other operations include maintaining database and generating reports at regular intervals.

Major software functions identified

1. User interface
2. Database management
3. Report generation

For user interface

S_{opt} : 1800
S_m :2000
S_{pess} : 4000

EV for user interface

EV = (1800 + 4*2000 + 4000) / 6
EV = 2300

For dababase managemant

S_{opt} : 4600
S_m : 6900
S_{pess} : 8600

EV for database management

EV = (4600 + 4*6900 + 8600) / 6
EV = 6800

For report generation

S_{opt} : 1200
S_m : 1600
S_{pess} : 3200

EV for Report generation

EV = (1200 + 4*1600 + 3200) / 6
EV = 1800

Function	Estimated LOC
User interface	2300
Database management	6800
Report generation	1800
Estimated LOC	10900

Suppose,
productivity = 3000 LOC / person-month
Development Cost = $8000 person-month

Person-month = LOC / Productivity
= 10900/3000
= 3.6
Cost of software = Development Cost * person-month
= 8000 * 3.6
= 29120
Cost = Cost of software ($) / LOC
= 29120 / 10900
= 2.67 $ / LOC

FP Estimation

Function oriented metrics focus on program "functionality" or "utility". Albrecht first proposed function point method, which is a function oriented productivity measurement approach.

Five information domain characteristics are determined and counts for each are provided and recorded in a table.

- Number of user inputs
- Number of user outputs
- Number of user inquires
- Number of files
- Number of external interfaces

Once the data have been collected, a complexity value is associated with each count. Each entry can be simple, average or complex. Depending upon these complexity values is calculated.
To compute function points, we use

$$FP = count\text{-}total \times [0.65 + 0.01 * SUM(F_i)]$$

F*i* (I= 1 to 14) are complexity adjustment values based on response to questions (1-14) given below. The constant values in the equation and the **weighing factors** that are applied to information domain are determined empirically.
F*i*

1. Does the system require reliable backup and recovery?
2. Are data communications required?

3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require on-line entry?
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?
8. Are the inputs, files, or inquires complex
9. Is the internal processing complex?
10. Is the code designed to be reusable?
11. Are masters files updated online?
12. Are conversation and installations included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

Rate each **factor** on a scale of 0 to 5

- 0 – No influence
- 1 – Incidental
- 2 – Moderate
- 3 – Average
- 4 – Significant
- 5 – Essential

Count-total is sum of all FP entries.
Once functional point has been calculated, productivity, quality, cost, and documentation can be evaluated.

Productivity = FP / person-month
Quality = defects / FP

Cost = $ / FP
Documentation = pages of documentation / FP

Example:

Considering the following information for a software development project to answer the question given below-

Same as LOC

Information Domain Values	Simple	Average	Complex	Count	FP Count
Number of inputs	4	10	16	10	40=(4*10)
Number of outputs	4	5	16	8	40=(5*8)
Number of inquiries	4	12	19	12	48=(4*12)
Number of files	3	6	10	6	60=(10*6)
Number of external interfaces	2	7	3	2	14=(7*2)
				Count- Total=	202=(40+40+48+60+14)

183 Complexity weighing factors are determined and the following results are obtained.
184

Factors (F*i*)	Value
Backup and recovery	4
Data Communication	1
Distributed processing	0
Performance critical	3
Existing operating environment	2
On-line data entry	5
Input transaction over multiple screens	5
Master file updated online	3
Information domain values complex	3
Internal processing complex	2
Code designed for reuse	0
Conversion / installation in design	1
Multiple installation	3
Application designed for change	5
Sum(F*i*)	37

185

Estimated number of FP:

$$FP_{estimated} = count\text{-}total * [0.65 + .01 * SUM (Fi)]$$

= 202 * [0.65 + (.01 * 37)]
= 202 * [0.65 + 0.37]
= 202 * 1.02
= 206.04

For historical data, productivity is 55.5 FP person-month and development cost is $8000 per month.

Productivity = FP / person-month
FP = 206.04 / 55.5
= 4 person-month

Total cost = Development cost * person-month
= $8000 * 4
= $32000
Cost per FP = Cost / FP
= $32000/206
= $155.34 per FP

Empirical Estimation

Resource models consist of one or more empirically derived equations. These equations are used to predict effort (in person-month), project duration, or other pertinent project data. There are four classes of resource models:

- Static single-variable models
- Static multi-variable models
- Dynamic multi-variable models
- Theoretical models

The basic version of the **CO**nstructive **CO**st **MO**del or COCOMO is an example of a static single-variable model. Barry Boehm introduced COCOMO models.
There is a hierarchy of these models.

COCOMO applies to three classes of software projects:

- Organic projects - "small" teams with "good" experience working with "less than rigid" requirements
- Semi-detached projects - "medium" teams with mixed experience working with a mix of rigid and less than rigid requirements
- Embedded projects - developed within a set of "tight" constraints. It is also combination of organic and semi-detached projects.(hardware, software, operational, ...)

The basic COCOMO equations take the form:

$E = a_b * (KLOC)\ b^b$

$\mathbf{D = c_b * (E)^{d_b}}$

Effort Applied (E) = $a_b(KLOC)^{b_b}$ **[person-months]**

Development Time (D) = $c_b(\text{Effort Applied})^{d_b}$ **[months]**

People required (P) = Effort Applied / Development Time **[count]**

Where, **KLOC** is the estimated number of delivered lines (expressed in thousands) of code for project. The coefficients a_b, b_b, c_b and d_b are given in the following table:

Project class	a_b	b_b	c_b	d_b
organic	2.4	1.05	2.5	0.38
semidetached	3.0	1.12	2.5	0.35
embedded	3.6	1.20	2.5	0.3

Example:

Use the information given in the table below

Project Class	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.38
Embedded	3.6	1.20	2.5	0.12

Assume that the expected size of an organic software project is 45.6 KLOC. Using the coefficients in the above table and the BASIC COCOMO MODEL, calculate the recommended number of people needs to be engaged to complete the project.

Effort Applied **(**E) = a_b * (KLOC) b^b
= 2.4 * $(45.6)^{1,05}$

= 132.47

Development Time (D) = $C_b * (E)^{d_b}$

= $2.5 *(132.47)^{0.38}$

= 2.5 * 6.4

= 16

People Required (P) = E/D

= 132.47/16

= 8.27

= 9 people

The Intermediate Cocomo formula takes the form:

$$E=a_i(KLoC)^{(b_i)}.EAF$$

Where E is the effort applied in person-months, **KLOC** is the estimated number of thousands of delivered lines of code for the project, and **The product of all effort multipliers results in an *effort adjustment factor (EAF)*.**The coefficient a_i and the exponent b_i are given in the next table. And the ci value is always 2.5.

Software project	a_i	b_i
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

The Development time **D** calculation uses **E** in the same way as in the Basic COCOMO.

Example:

Consider an organ project where three module to implement. User interfaces 0.10 KLOC, database management 0.8 KLOC and report generation 0.18 KLOC. Efforts and project class value as follows:

Software project Class	a_i	b_i	d_i
Organic	3.2	1.05	1.05
Semi-detached	3.0	1.12	1.12
Embedded	2.8	1.20	1.20

Cost Drivers	Level	Effort Adjustment Factor
Complexity	High	1.15
Storage	High	1.06
Experience	Low	1.13
Programming Capabilities	Low	1.17
Analyst capability	High	1.23

Memory Constraints	Low	1.01

Calculate the effort applied in person-month using **Intermediate COCOMO Model**. What will be total *development time* and how many *people* should be hired?

Solution:

Effort (E) = a_i**(KLOC)**$^{(b_i)}$**EAF**

$= 3.2\ (0.10 + 0.8 + 0.18)^{1.05} * (1.15* 1.06* 1.13* 1.17*1.23*1.01)$

$= 3.2 * 1.08^{1.05} * 2.002$

= 6.945

Development Time, $\mathbf{D = C_b * (E)^{d_b}}$

$\mathbf{= 2.5 * 6.945^{1.05}}$

= 19.129

People, N = E/D = 6.945/19.129 = 0.36